

Software Engineering Report
(CST2550 Group Coursework)

Team Members:

Kushmandaa Devi Bhaugeeruth -M00965269

Veerapen Leynah Reeya-M00976615

Aba Nuni Ssenyange - M00956871

Kritisha Sunjhoreea-M01014126

Project Title:

Gym Management System

Submission Date:

April 16, 2025

Lecturers:

- Aisha Idoo
- Karel Veerabudren

1. Introduction

The Gym Hive LTD is a desktop-based software solution designed to simplify and automate the operations of a fitness center. Built using C# in Visual Studio with database integration (e.g., SQL Server), this system supports essential gym functionalities like member registration, trainer dashboards, member detail management, and workout schedules.

The Gym Management System project is designed to support the operations of a gym by managing member records, scheduling, and fitness program data. Unlike basic record-keeping systems, this application integrates custom data structures for efficient search, insertion, and removal of data related to gym memberships and session bookings. The project employs a fully self-developed solution in C# using the .NET framework (MS C# .NET) without resorting to any third-party libraries, ensuring that every component—data structure, algorithm, and user interface—is developed in-house.

The report is organized as follows:

- The Design section justifies the selection of data structures and explains the key algorithms using pseudo-code and complexity analysis.
- The Testing section describes the approach taken to validate the implementation, including a table of representative test cases.
- The Conclusion summarizes the work accomplished, discusses limitations, and provides recommendations for future improvements.
- The References section lists all supporting documents and sources used in Harvard style.

2. Design and Implementation

2.1 Software Planning and Project Management

We began by setting up a Github repository to manage our source code and documentation. Our team used a makefile to automate builds and maintained detailed sprint plans and meeting minutes to track progress. Team roles were clearly designated: a Scrum Master to coordinate efforts, a secretary to manage administrative tasks, two developers to implement functionalities, and one tester to validate operations through rigorous unit testing.

Given that the coursework prohibits the use of third-party libraries and STL containers, our team designed a custom data structure tailored for efficient storage and search operations. We chose a balanced binary search tree (BST) as the underlying mechanism because:

- **Efficiency:** The BST allows for average-case logarithmic performance for insertion, deletion, and search operations, which is crucial for handling large datasets.
- **Customizability:** A self-designed structure provides flexibility to integrate project-specific functionalities and control over memory management.

- **Algorithmic Clarity:** A BST's properties allow us to clearly demonstrate time complexity in our analysis, aligning with the assignment's requirement to analyze the core operations.

In parallel, we integrated SQL to handle data persistence. Our system uses a SQL database to store and manage records persistently. This approach allows us to combine the speed of in-memory data structures (BST) with the reliability and durability of SQL databases.

2.2. Algorithm Design and Pseudocode

Below is a simplified pseudocode for the key BST operations along with pseudocode for the SQL interactions.

Insertion (Add Data Element) – BST Operation:

```

FUNCTION Insert(root, data)      IF root IS NULL THEN      CREATE
new Node(data)                  RETURN new Node              IF data < root.value THEN
root.left = Insert(root.left, data)  ELSE                  root.right = Insert(root.right, data)
RETURN root

```

Deletion (Remove Data Element) – BST Operation:

```

FUNCTION Delete(root, data)      IF root IS NULL THEN
RETURN NULL                      IF data < root.value THEN      root.left = Delete(root.left,
data)                      ELSE IF data > root.value THEN      root.right = Delete(root.right, data)
ELSE                          IF root.left IS NULL THEN      RETURN root.right
ELSE IF root.right IS NULL THEN      RETURN root.left      temp =
FindMin(root.right)      root.value = temp.value      root.right =
Delete(root.right, temp.value)      RETURN root

```

Search (Lookup Data Element) – BST Operation:

```

FUNCTION Search(root, data)      IF root IS NULL OR root.value == data
THEN      RETURN root      IF data < root.value THEN      RETURN
Search(root.left, data)      RETURN Search(root.right, data)

```

SQL Database Integration – Read/Write Operations:

To persist data and enable retrieval across sessions, our system interfaces with a SQL database. The following pseudocode outlines our basic SQL operations:

Insert Record to SQL Database:

```

FUNCTION InsertRecord(data)      SQL_QUERY = "INSERT INTO Records
(value, otherColumns) VALUES (data.value, data.otherValues);"
Execute(SQL_QUERY)

```

Delete Record from SQL Database:

```

FUNCTION DeleteRecord(data)      SQL_QUERY = "DELETE FROM
Records WHERE value = data.value;"      Execute(SQL_QUERY)

```

Retrieve All Records from SQL Database and Populate BST:

```

FUNCTION LoadDataIntoBST()      SQL_QUERY = "SELECT * FROM

```

```

Records;"      resultSet = ExecuteQuery(SQL_QUERY)      FOR EACH record IN
resultSet DO      bstRoot = Insert(bstRoot, record.value)      RETURN bstRoot

```

This dual approach—managing data in-memory with a BST for rapid operations and persisting data in a SQL database for durability—ensures that our system maintains high performance while safeguarding against data loss.

2.3. Time Complexity Analysis

For a balanced BST implementation, the key operations have the following time complexities:

- Insertion: $O(\log n)$ on average, but $O(n)$ in the worst case if the tree degenerates.
- Deletion: $O(\log n)$ on average, with the worst-case scenario again $O(n)$ when re-balancing is needed or the tree becomes skewed.
- Search: $O(\log n)$ on average, and $O(n)$ in the worst-case scenario.

For the SQL operations, execution times depend on the database engine and the structure of the SQL queries. Generally, well-indexed SQL operations can be performed in $O(\log n)$ time for searches, while insertions and deletions are often near $O(1)$ to $O(\log n)$ under optimal conditions. Our design ensures that the performance of the in-memory BST mitigates the latency introduced by SQL operations, thus balancing rapid data manipulation with persistent storage.

Summary

By combining a custom-balanced BST with SQL database operations, our design leverages the strengths of both approaches. The BST provides efficient, in-memory operations and clear complexity analysis while the SQL component ensures long-term data persistence, robust data handling, and easier data management through standard database operations. This integrated approach aligns with our coursework requirements and supports both high performance and data reliability.

3. Testing

3.1 Statement of Testing Approach

To ensure that both our in-memory data structure (custom balanced BST) and the SQL database interactions operate as intended, we adopted a rigorous, multi-phase testing approach using unit testing frameworks (MSTest or NUnit):

- Unit Testing for Core Functionality:
Each fundamental operation (insertion, deletion, search) in the BST was subjected to unit tests. These tests verified that, for any given input, the data structure responds correctly. This phase ensured the tree maintained its properties after every operation
- Boundary and Edge Case Testing:
We designed tests to handle edge conditions—such as inserting into an empty tree,

handling duplicate entries, and ensuring correct behavior when searching for or deleting non-existent nodes. Testing these scenarios helps guarantee that the application behaves robustly even under unexpected inputs.

- **Integration Testing with SQL Database:**
Since the application features both an in-memory BST and persistent storage via SQL, we implemented integration tests to validate that SQL operations (insert, delete, read) and BST operations remain synchronized. For example, after an SQL record insertion, the BST is repopulated correctly with the new data, ensuring consistency.
- **Performance Testing:**
We observed the response times of BST operations and SQL queries in realistic scenarios. These tests confirmed that performance remained within acceptable limits (average-case logarithmic complexity for BST operations and efficient SQL queries), thereby validating our design choices.
- **User Scenario Testing:**
Finally, we simulated user interactions by running end-to-end test scenarios. This included menu-driven operations where the user requests an addition, search, or deletion, with corresponding changes reflected both in the SQL database and in-memory structure.

3.2 Table of Test Cases

Below is an example table of test cases covering both the BST functionality and SQL integration:

Test Case ID	Description	Input Data	Expected Outcome	Comments
TC-01	Insert into empty BST	Insert a single node with value 15	BST contains one node; the root is correctly assigned	Validates basic insertion
TC-02	Insert duplicate values	Insert two nodes with value 15	BST handles duplicates as per defined business logic	Ensure duplicate handling or rejection
TC-03	Search for existing element	Search for the node with value 15	Returns the node with value 15	Verifies correctness of search operation
TC-04	Search for non-existing element	Search for value 20 in BST with no 20	Returns NULL or an indication of "element not found"	Checks proper handling of non-existent elements
TC-05	Delete a leaf node	Delete a node that has no children	Node is removed; BST structure remains valid	Validates removal for simplest case
TC-06	Delete a node with one child	Delete a node with only one child	Child node is correctly promoted; BST remains ordered	Ensures pointer reassignments are handled

Test Case ID	Description	Input Data	Expected Outcome	Comments
TC-07	Delete a node with two children	Delete a node with two children	Node is replaced with its in-order successor; tree remains balanced	Verifies complex deletion behavior
TC-08	SQL Insert operation	Insert a record with value 15 into SQL	Record is added in the SQL database; BST updated after reload	Tests SQL & in-memory integration
TC-09	SQL Delete operation	Delete a record with value 15 from SQL	Record is removed from SQL; corresponding BST node is deleted	Confirms consistency between SQL and BST
TC-10	SQL Read operation for BST population	Retrieve all records from SQL database	BST is populated with all records from the SQL database	Verifies data persistence and loading process

Summary

By implementing a comprehensive set of tests across both the standalone BST operations and the integrated SQL functionality, we ensure that the software meets performance, reliability, and correctness objectives. This thorough testing approach minimizes the likelihood of runtime errors and confirms that the system behaves predictably across all scenarios.

This section can be further tailored to your team's specific implementation and any additional test cases identified during the development process.

4. Conclusion

Summary of Work Done

Our project achieved the dual objectives of designing a custom data structure and integrating persistent data storage using SQL. We implemented a balanced binary search tree (BST) from scratch, including key operations such as insertion, deletion, and search. In parallel, we established SQL operations to handle data insertion, deletion, and retrieval, ensuring that our in-memory BST and persistent storage remain synchronized. Comprehensive testing—encompassing unit, integration, performance, and user scenario tests—was conducted to verify functionality and maintain performance standards.

Limitations and Critical Reflection

Several limitations were encountered during this project:

- **BST Imbalance:** Although a balanced BST was our goal, our implementation could suffer from skewing in worst-case scenarios, which may degrade performance over time.

- **Error Handling:** The robustness of error detection and handling in both the BST and SQL interactions requires further enhancement, as current strategies do not fully address unexpected runtime issues or data discrepancies.
- **Scalability Constraints:** The design, while efficient for moderate data sizes, may require additional optimization for handling significantly larger datasets, particularly with respect to ensuring SQL query efficiency.
- **Resource Constraints:** Time and resource limitations meant that we could not integrate more complex self-balancing mechanisms or advanced database optimization strategies, which would further enhance system reliability.

Future Improvements and Changes in Approach

To avoid repeating these issues in future projects, the following adjustments are recommended:

- **Utilize Self-Balancing BST Variants:** Incorporating AVL or Red-Black Tree implementations would offer more reliable balancing and maintain logarithmic operation times, even under worst-case inputs.
- **Enhanced Error Handling:** Future iterations should focus on robust error detection and comprehensive exception handling across both in-memory and database components, ensuring that operations remain fault-tolerant.
- **Performance Optimization:** Investing in more detailed performance profiling can help identify bottlenecks in SQL queries and BST operations, guiding targeted optimizations.
- **Iterative Improvement and Refactoring:** A more iterative development cycle with ongoing performance reviews and code refactoring will ensure that each component evolves to meet scalability and robustness requirements as the dataset grows.